ProblemSet03_Ex3.R

cyrilgabriele

2026-05-04

```r
############################
# Load packages
############################
library(YieldCurve)
```

```
## Loading required package: xts

## Loading required package: zoo

##
## Attaching package: 'zoo'

## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric
```

```r
set.seed(13)

############################
# Prepare data
############################

yc.data <- read.csv2("YieldCurve.csv", header = TRUE, stringsAsFactors = FALSE)

# Extract chosen yield curve column
# Adjust column index if your YieldCurve.csv uses another column.
yields <- as.numeric(yc.data[, 2])

cat("
Exercise 3: Short-Rate Realized Volatility Bonus Product

Product description.

The product is a five-year Short-Rate Realized Volatility Bonus Product.
It pays a positive amount at maturity if the realized annualized volatility
of the Hull-White short rate exceeds a fixed volatility strike. Economically,
the product is attractive to an investor who wants exposure to unusually large
movements in short rates, independently of whether rates move upward or downward.

Model assumptions.
```

```
The short rate follows the Hull-White model under the traditional risk-neutral
measure Q*. I use alpha = 0.20, sigma = 0.01, 1000 Monte Carlo paths, quarterly
time steps, maturity T = 5 years, notional N = 10,000,000, and volatility strike
K_vol = 0.0075.

Formal payoff.

Let 0 = t_0 < t_1 < ... < t_n = T be the quarterly simulation grid and let r_t
be the simulated short rate. Define

RV_T = sqrt((1 / T) * sum_{i=1}^n (r_{t_i} - r_{t_{i-1}})^2).

The payoff at maturity T is

X_T = N * max(RV_T - K_vol, 0).

The time-0 price under Q* is estimated by

V_0 = E^{Q*}[exp(-sum_{i=0}^{n-1} r_{t_i} * Delta t) * X_T].

Result.

The Monte Carlo price is 24,858.77 with standard error 422.70.

")
```

```
##
## Exercise 3: Short-Rate Realized Volatility Bonus Product
##
## Product description.
##
## The product is a five-year Short-Rate Realized Volatility Bonus Product.
## It pays a positive amount at maturity if the realized annualized volatility
## of the Hull-White short rate exceeds a fixed volatility strike. Economically,
## the product is attractive to an investor who wants exposure to unusually large
## movements in short rates, independently of whether rates move upward or downward.
##
## Model assumptions.
##
## The short rate follows the Hull-White model under the traditional risk-neutral
## measure Q*. I use alpha = 0.20, sigma = 0.01, 1000 Monte Carlo paths, quarterly
## time steps, maturity T = 5 years, notional N = 10,000,000, and volatility strike
## K_vol = 0.0075.
##
## Formal payoff.
##
## Let 0 = t_0 < t_1 < ... < t_n = T be the quarterly simulation grid and let r_t
## be the simulated short rate. Define
##
## RV_T = sqrt((1 / T) * sum_{i=1}^n (r_{t_i} - r_{t_{i-1}})^2).
##
## The payoff at maturity T is
##
```

```
## X_T = N * max(RV_T - K_vol, 0).
##
## The time-0 price under Q* is estimated by
##
## V_0 = E^{Q*}[exp(-sum_{i=0}^{n-1} r_{t_i} * Delta t) * X_T].
##
## Result.
##
## The Monte Carlo price is 24,858.77 with standard error 422.70.
```

```r
# ================================================================
# Part 1
# Nelson-Siegel-Svensson Model
# ================================================================
```

```r
# Estimate NSS model
```

```r
maturity <- c(3/12, 6/12, 9/12, seq(1, 30, 1))

NSSmodel <- Svensson(yields, maturity)

# Svensson() returns tau1 and tau2, which are reciprocals of lambda1/lambda2
b0 <- NSSmodel[1]
b1 <- NSSmodel[2]
b2 <- NSSmodel[3]
b3 <- NSSmodel[4]

lambda1 <- 1 / NSSmodel[5]
lambda2 <- 1 / NSSmodel[6]

cat("\nNSS parameter estimates\n")
```

```
##
## NSS parameter estimates
```

```r
cat(sprintf("b0      : %.6f\n", b0))
```

```
## b0      : 1.497137
```

```r
cat(sprintf("b1      : %.6f\n", b1))
```

```
## b1      : 0.423610
```

```r
cat(sprintf("b2      : %.6f\n", b2))
```

```
## b2      : 1.651342
```

```r
cat(sprintf("b3      : %.6f\n", b3))
```

```
## b3      : 6.602684
```

```r
cat(sprintf("lambda1 : %.6f\n", lambda1))
```

```
## lambda1 : 1.434619
```

```r
cat(sprintf("lambda2 : %.6f\n", lambda2))
```

```
## lambda2 : 0.065210
```

```r
# Spot yield curve function y0(T)
```

```r
# y0(T) returns DECIMAL rates, e.g. 0.025 for 2.5%.
y0 <- function(T){
  F1 <- (1 - exp(-lambda1 * T)) / (lambda1 * T)
  F2 <- (1 - exp(-lambda1 * T)) / (lambda1 * T) - exp(-lambda1 * T)
  F3 <- (1 - exp(-lambda2 * T)) / (lambda2 * T) - exp(-lambda2 * T)

  (b0 + b1 * F1 + b2 * F2 + b3 * F3) / 100
}
```

```r
# Arbitrage-free forward rate function f0(T1,T2)
```

```r
# f0(T1,T2) returns DECIMAL rates.
f0 <- function(T1, T2){
  (y0(T2) * T2 - y0(T1) * T1) / (T2 - T1)
}

# T_1 <- 1
# T_2 <- seq(T_1 + 0.01, 30, by = 0.01)
# f0_curve <- f0(T_1, T_2)
```

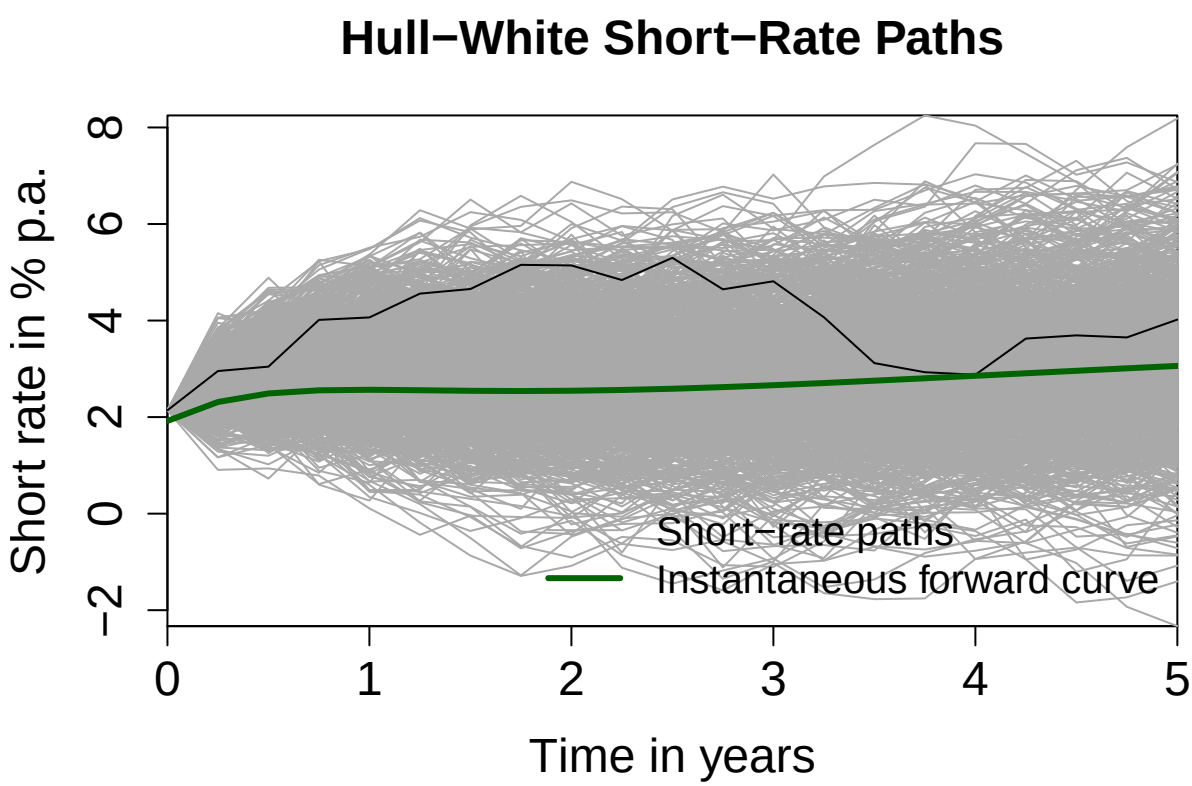
```r
# Instantaneous forward rate function f0inst(T)
```

```r
# f0inst(T) returns DECIMAL rates.
f0inst <- function(T){
  F1 <- exp(-lambda1 * T)
  F2 <- exp(-lambda1 * T) * lambda1 * T
  F3 <- exp(-lambda2 * T) * lambda2 * T

  (b0 + b1 * F1 + b2 * F2 + b3 * F3) / 100
}

# T_grid <- seq(0.25, 30, by = 0.01)
# f0inst_curve <- f0inst(T_grid)

# ================================================================
# EXERCISE 2
# Hull-White Model
# ================================================================
```

```r
# Simulate Hull-White short-rate paths
```

```r
paths <- 1000
alpha <- 0.20
sigma <- 0.01
T <- 5              # Maturity T

# quarterly increments for the MC simulation
s <- 4

dt <- rep(1 / s, T * s)
t <- c(0, cumsum(dt))

# Initial short rate: very short maturity spot rate
r0 <- y0(1 / s)

# Derivative of instantaneous forward curve
df0inst_dt <- function(T){
  (
    -lambda1 * exp(-lambda1 * T) * (b1 - b2 + b2 * lambda1 * T) +
      lambda2 * exp(-lambda2 * T) * (b3 - b3 * lambda2 * T)
  ) / 100
}

# Simulate short-rate paths -- LOOP version
rt <- matrix(NA, nrow = paths, ncol = length(t))
rt[, 1] <- r0

for(j in 1:paths){
  for(i in 2:length(t)){

    # Time-varying mean-reversion level
    theta <- df0inst_dt(t[i - 1]) + alpha * f0inst(t[i - 1])

    # Wiener increment: detla_W ~ N(0, delta_t)
    dW <- rnorm(n = 1, mean = 0, sd = sqrt(dt[1]))

    # Euler discretization of Hull-White:
    # r_t = r_{t-dt} + (theta_t - alpha*r_{t-dt})*dt + sigma*dW
    rt[j, i] <- rt[j, i - 1] +
      (theta - alpha * rt[j, i - 1]) * dt[1] +
      sigma * dW
  }
}

# Plot simulated short-rate paths with instantaneous forward curve
short_rate_ylim <- range(c(rt, f0inst(t)), na.rm = TRUE) * 100

plot(t, rt[1, ] * 100,
     type = "l",
     lwd = 1,
     xlim = range(t),
     ylim = short_rate_ylim,
```

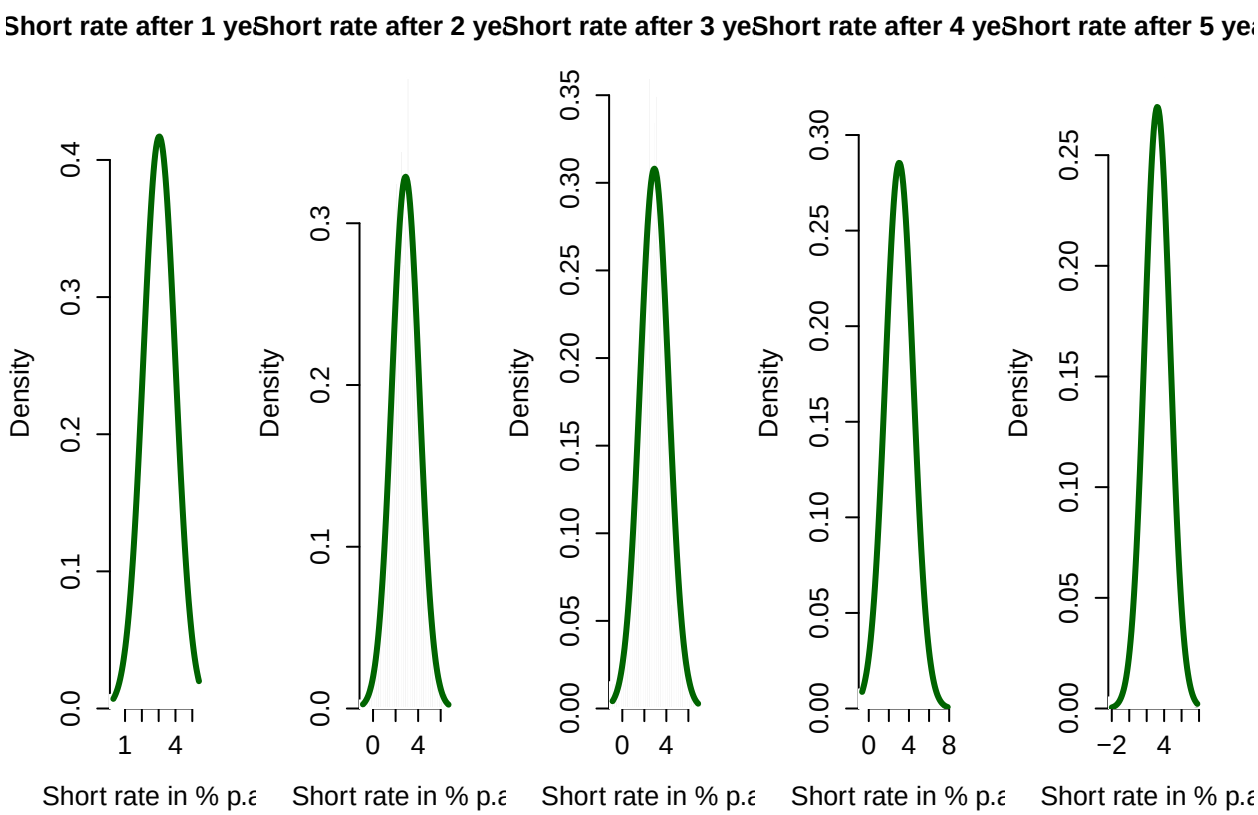
```r
     xaxs = "i",
     yaxs = "i",
     col = "darkgrey",
     cex.axis = 1.5,
     cex.lab = 1.5,
     cex.main = 1.5,
     main = "Hull-White Short-Rate Paths",
     xlab = "Time in years",
     ylab = "Short rate in % p.a.")

for(j in 2:paths){
  lines(t, rt[j, ] * 100,
        lwd = 1,
        col = "darkgrey")
}

lines(t, rt[1, ] * 100,
      lwd = 1,
      col = "black")

lines(t, f0inst(t) * 100,
      lwd = 3,
      col = "darkgreen")

legend("bottomright",
       legend = c("Short-rate paths", "Instantaneous forward curve"),
       lty = 1,
       lwd = c(1, 3),
       col = c("darkgrey", "darkgreen"),
       bty = "n",
       cex = 1.25)
```




```
# Histograms after 1 to 5 years
```

```
years <- c(seq(1, T))

year.idx <- years * s + 1
rt.selected <- rt[, year.idx]

colnames(rt.selected) <- paste0("t = ", years)

par.old <- par(no.readonly = TRUE)
par(mfrow = c(1, 5))

for(i in seq_along(years)){

  rt_i <- rt.selected[, i]

  mu_i <- mean(rt_i)
  sd_i <- sd(rt_i)

  hist(rt_i * 100,
       breaks = 40,
       probability = TRUE,
       xaxs = "i",
       yaxs = "i",
       col = "lightgrey",
       border = "white",
```

```
        main = paste("Short rate after", years[i], "year(s)"),
        xlab = "Short rate in % p.a.",
        ylab = "Density",
        cex.axis = 1.2,
        cex.lab = 1.2,
        cex.main = 1.2)

  curve(dnorm(x, mean = mu_i * 100, sd = sd_i * 100),
        add = TRUE,
        col = "darkgreen",
        lwd = 3)
}
```




```
par(par.old)

rt.summary <- data.frame(
  year = years,
  mean_decimal = colMeans(rt.selected),
  sd_decimal = apply(rt.selected, 2, sd),
  mean_percent = colMeans(rt.selected) * 100,
  sd_percent = apply(rt.selected, 2, sd) * 100
)

cat("\nShort-rate summary\n")
```

```
##
## Short-rate summary
```

```r
print(rt.summary)
```

```
##       year mean_decimal  sd_decimal mean_percent sd_percent
## t = 1    1   0.03037212 0.009563471     3.037212  0.9563471
## t = 2    2   0.02886390 0.012133353     2.886390  1.2133353
## t = 3    3   0.02909509 0.012948168     2.909509  1.2948168
## t = 4    4   0.03034108 0.013973072     3.034108  1.3973072
## t = 5    5   0.03223674 0.014676149     3.223674  1.4676149
```

```r
# Expected future zero-coupon bond price
```

```r
# Initial zero-coupon bond price from today's curve
P0 <- function(T){
  exp(-y0(T) * T)
}

# Hull-White B(t,T)
B <- function(t_bond, T_bond){
  (1 - exp(-alpha * (T_bond - t_bond))) / alpha
}

# Hull-White ln A(t,T)
ln_A <- function(t_bond, T_bond){
  log(P0(T_bond) / P0(t_bond)) +
    B(t_bond, T_bond) * f0inst(t_bond) -
    sigma^2 / (4 * alpha) *
    (1 - exp(-2 * alpha * t_bond)) *
    B(t_bond, T_bond)^2
}

# Hull-White log zero-coupon bond price
ln_P_tilde_t <- function(t_bond, T_bond, r_t){
  ln_A(t_bond, T_bond) -
    B(t_bond, T_bond) * r_t
}
```

```r
# Pricing of the Short-Rate Realized Volatility Bonus Product:
# Done under traditional risk-neutral measure Q*
```

```r
# REMARK: T and s set above since already needed for the simulation
#         of the Hull-White short-rate paths

# self defined assumptions/definitions
notional <- 10000000
K_vol = 0.0075                # volatility strike = 0.75%

# Index of payoff date
idx_pay <- T * s + 1   # with s = 4, this gives 21
```

```r
# Short-rate paths from time 0 to product maturity T
rt_product <- rt[, 1:idx_pay]

# Quarterly short-rate changes:
# delta_r[j, i] = r_j(t_i) - r_j(t_{i-1})
delta_r <- rt_product[, -1] - rt_product[, -ncol(rt_product)]

# Short-rate paths (= 1000 ; set above) from time 0 to time T_product
rt_product <- rt[, 1:idx_pay]

# Realized annualized short-rate volatility per path
RV_T <- sqrt(rowSums(delta_r^2) / T)

# Payoff at T_product
payoff_T <- notional * pmax(RV_T - K_vol, 0)

# Pathwise discount factor from 0 to T_product under Q*
DF_0_T <- exp(
  -rowSums(rt_product[, 1:(ncol(rt_product) - 1), drop = FALSE]) * dt[1]
)

# Monte Carlo price
volatility_bonus_price <- mean(DF_0_T * payoff_T)

# Monte Carlo standard error
volatility_bonus_price_se <- sd(DF_0_T * payoff_T) / sqrt(paths)

cat("\nShort-Rate Realized Volatility Bonus Pricing under Q*\n")
```

```
##
## Short-Rate Realized Volatility Bonus Pricing under Q*
```

```r
cat(sprintf("Expected realized volatility: %.6f\n", mean(RV_T)))
```

```
## Expected realized volatility: 0.010373
```

```r
cat(sprintf("Price today: %.6f\n", volatility_bonus_price))
```

```
## Price today: 24858.770339
```

```r
cat(sprintf("Monte Carlo SE: %.6f\n", volatility_bonus_price_se))
```

```
## Monte Carlo SE: 422.704705
```